DIA 2 · TERÇA-FEIRA · 14 ABR

20h – 23h · BRASÍLIA

Contextualizar

De “a IA inventa” para “a IA pesquisa nos seus dados e responde certo”

Instrutor: Luan Moreno

AIDE BRASIL · aide-brasil.ai

Context Engineering · RAG · Ledger · MCP · LlamaIndex

Context is the new prompt. What you give the model defines what the model can do.

— ANDREJ KARPATHY, 2026

RECAP · DIA 1

O Que Construímos Ontem

7

Blocos ao Vivo

DOCKER · SHADOWTRAFFIC · CLAUDE CODE

Pipeline local rodando

Dados fluindo via ShadowTraffic + Postgres + Qdrant. AI Coding Agents. Agentic Commerce.

Pydantic

+ Structured Outputs

VALIDAÇÃO · TIPAGEM · JSON

Schema como contrato

Validação tipada com Pydantic e Claude retornando JSON estruturado.

Ledger

+ Memory

CONCEITO APRESENTADO NO DIA 1

Hoje ganham vida →

SQL exato + busca semântica. Dois stores apresentados ontem, construídos hoje.

COMMUNITY · DIA 1

A Voz da Comunidade

200+

Guerreiros até a 1h

5 HORAS DE SESSÃO

Engajamento recorde

200 participantes ainda online depois da meia-noite. Vocês são insanos.

50+

Vezes que falaram FODA

PICO: DEMO AO VIVO

ShopAgent + dados reais

O momento que o Claude consultou dados ao vivo do Zoom explodiu o chat.

#1

Pergunta da Noite

KB VS RAG VS SKILL VS MCP

Resolvemos hoje →

15+ pessoas fizeram a mesma pergunta. A confusão morre nesta aula.

1500+

Mensagens no Chat

LINKEDIN · LIVROS · DÚVIDAS

Comunidade ativa

Networking, recomendações de livros e debates técnicos até a madrugada.

PERGUNTAS REAIS · RESPOSTAS HOJE

Suas Dúvidas → Nossa Agenda

"Qual a diferença entre KB, RAG, Skill e MCP?"
15+ pessoas perguntaram

"A IA inventa dados quando pergunto faturamento"
Problema universal sem dados reais

"Como encontrar clientes insatisfeitos em 50K reviews?"
SQL não resolve — precisa semântica

"Como o agente decide sozinho qual store usar?"
O teaser que acendeu a curiosidade

Bloco 5: RAG vs Ledger
Cada conceito no seu lugar. KB ≠ RAG ≠ Skill ≠ MCP.

Context Eng · RAG · Ledger

Bloco 6: The Ledger
SQL exato via MCP Supabase. Faturamento nunca mais inventado.

SQL · MCP Supabase

Bloco 7: The Memory
Busca semântica no Qdrant. Sinônimos, sentimento, contexto.

Qdrant · Semântica

Demo: Dual-Store + Provocação
Vocês escolhem o store. Amanhã, o agente escolhe sozinho.

Demo · Dia 3 Preview

DIA 2 · TERÇA-FEIRA · CONTEXTUALIZAR

A Progressão da Noite

Do prompt ingênuo à inteligência contextual — passo a passo

AGENDA DA NOITE

4 Blocos, 1 Missão

01
FUNDAMENTOS

Prompt Eng + Context Eng + MindSet

20h00–20h40

→

02
ARQUITETURA

RAG vs Ledger + Memory Types + Roles

20h40–21h30

→

03
CONSTRUÇÃO

LlamaIndex + Qdrant + Postgres + MCP

21h30–22h30

→

04
DEMONSTRAÇÃO

Claude Code pesquisa nos dois stores

22h30–23h00

ARCO DA SEMANA

Autonomia Progressiva

1 — DIA 1

"Crie o schema do e-commerce"

Claude Code gera tabelas, FKs e validação com **Pydantic**

ShadowTraffic · Pydantic

2 — HOJE

"Qual faturamento do Sudeste? Quem reclama de entrega?"

A IA pesquisa no **Ledger** *(SQL) e no* **Memory** *(Qdrant) via MCP – sem inventar*

RAG · Ledger · MCP

3 — DIA 3

"Analise os clientes insatisfeitos"

O agente decide sozinho: **Qdrant** *para sentimento,* **Postgres** *para números*

LangChain · ReAct

4 — DIA 4

"Relatório executivo com dashboard"

3 agentes especializados + frontend profissional gerado por **IA**

CrewAI · Frontend

BLOCO 1 · MINDSET

O Engenheiro AI-Native

01

MODELO DE CONHECIMENTO

O Profissional E-Shape

De coder a orquestrador — combina Product, Design Thinking e AI Literacy na barra horizontal, com 3 profundidades verticais que fazem a diferença.

Broad Skills — A Barra do E

- Product Management
- Design Thinking
- AI Literacy

Engineering

DEEP SKILL 1

- Backend & Data Pipelines
- SQL, Python, Pydantic
- Testing & Code Review
- Docker & Cloud

MUDANÇA DE PAPEL
De escrever código a editar e revisar

PROFUNDIDADE

AI-Native Tooling

DEEP SKILL 2

- RAG & Embeddings
- Agent Orchestration
- Context Engineering
- Prompt & Evals

MUDANÇA DE PAPEL
De "como codar" a "como orquestrar"

PROFUNDIDADE

Architecture

DEEP SKILL 3

- System Design & Trade-offs
- Guardrails & Quality
- Ledger + Memory Pattern
- MCP & Multi-Agent

MUDANÇA DE PAPEL
De implementar a definir guardrails

PROFUNDIDADE

De coder a orchestrator — a IA é o agente, você é o arquiteto

broad skills × deep prongs = AI-native engineer

OS 3 PILARES

Pilares do Engenheiro *AI-Native*

Fundamentos

A BASE

SQL exato, Python limpo, cloud, schema. A IA não substitui – amplifica. Sem fundamentos sólidos, o agente herda suas fraquezas.

- ✓ *SQL: JOINs, window functions, CTEs*
- ✓ *Python: tipos, Pydantic, async*
- ✓ *Cloud: Docker, Postgres, Qdrant*

NA PRÁTICA

Ontem vocês construíram o schema do ShopAgent. Cada tabela, cada FK, cada tipo – isso é fundamento.

A Base

SQL PYTHON CLOUD

A IA amplifica – mas não cria fundamentos do zero

Comunicação

O CONTRATO

Traduzir requisito de negócio em spec técnico preciso. O prompt é o contrato entre você e a máquina.

- ✓ *Requisitos claros → prompts claros*
- ✓ *Schema como contrato de interface*
- ✓ *Tradução negócio ↔ técnico*

NA PRÁTICA

"Faturamento de março" vira "SELECT SUM(total) FROM orders WHERE created_at BETWEEN..." – a tradução é o skill.

O Contrato

PROMPT SPEC TRADUÇÃO

Quem escreve bem, prompta bem – comunicação é o skill oculto

Hardening

A GARANTIA

Validação, guardrails, evals. IA que não falha graciosamente não vai para produção. Este é o pilar que todos subestimam.

- ✓ *Pydantic: validação em runtime*
- ✓ *DeepEval: testes de qualidade LLM*
- ✓ *Guardrails: constraints no prompt*

NA PRÁTICA

Agente gera SQL errado? Pydantic rejeita. Resposta sem sentido? DeepEval flagra. Hardening é o que separa demo de produção.

A Garantia

GUARDRAILS EVALS FAIL-SAFE

O pilar que separa demo de produção – nunca subestime

BLOCO 2 · PROMPT ENGINEERING

A Base que Todos Subestimam

Técnicas provadas — e por que o conhecimento de domínio muda tudo

DOMÍNIO × PROMPT

O Domínio como *Amplificador*

A mesma pergunta, dois resultados opostos — a diferença é o ***contexto de domínio****.*

TÉCNICAS DE PROMPTING

6 Técnicas Provadas

Cumulativas, não exclusivas — use todas juntas. Cada uma resolve um tipo diferente de falha do modelo.

The hottest new programming language is English.

— ANDREJ KARPATHY, 2026

BLOCO 3 · CONTEXT ENGINEERING

O Coração de Qualquer Sistema

FUNDAMENTOS

4 Pilares do Context Engineering

Cada pilar alimenta o contexto do LLM. Remova um — a IA improvisa.

EVOLUÇÃO

Progressão: Prompt → Context → RAG → MCP

Cada fase resolve o problema da anterior. De "alucinação" a "resposta híbrida automática".

Fase 1

Prompt Engineering

Pergunto direto. Claude inventa quando não sabe.

▼ Alucinação

→

Fase 2

Context Engineering

Dou schema + exemplos. Claude entende estrutura. Mas dados mudam todo dia.

▲ Estrutura

→

Fase 3

RAG + Ledger

Recupero dados frescos antes de responder. Claude nunca inventa.

▲ Dados Frescos

→

Fase 4

MCP Conecta Tudo

Agente chama Postgres + Qdrant. Resposta híbrida automática.

★ Autonomia Total

ANTHROPIC ENGINEERING

Anatomia de uma Context Window

SYSTEM

System Prompt

*Instruções base, persona e regras do jogo. Define a **altitude** certa.*

"Right Altitude"

TOOLS

Tools & MCP

*Definições de tools, schemas e contratos MCP. **Minimal & clear**.*

Schemas + Contratos

EXAMPLES

Few-Shot

*Exemplos canônicos. **Diversos**, não exaustivos. Padrão > quantidade.*

"Worth 1000 Words"

DATA

Retrieved Data

***Just-in-time** retrieval. RAG + Ledger queries. Dados frescos.*

Dados Frescos

HISTORY

Msg History

*Histórico multi-turn. User + Assistant. **Compactável**.*

Compactável

MEMORY

Agent Memory

*Notas estruturadas. Persistência **externa** ao modelo.*

Long-Horizon

CONTEXT ROT

O Contexto Apodrece — Attention Budget

Precisão não cai de um penhasco — degrada gradualmente. Mais tokens = mais ruído no sinal. A atenção do transformer é n^2: cada token atende a todos os outros.

LONG-HORIZON AGENTS

3 Técnicas para Agentes que Pensam Longe

Como manter coerência em tarefas de horas? A Anthropic propõe três estratégias complementares.

EVIDÊNCIA

Context Engineering em Produção

Os números mostram: contexto bem estruturado é mais poderoso que modelo maior.

70%

STANFORD HAI 2025

▼ Erros evitáveis

Dos erros de LLM em produção vêm de contexto insuficiente, não de modelo ruim.

3x

ANTHROPIC INTERNAL

▲ Precisão comprovada

Melhoria na precisão ao adicionar schema como contexto explícito ao prompt.

$0

TRADE-OFF REAL

✓ Zero fine-tuning

Custo extra para Context Engineering vs. fine-tuning. Contexto bate fine-tune na maioria dos casos.

BLOCO 4 · MEMORY TYPES

Como a IA Lembra

04

ARQUITETURA DE MEMÓRIA

3 Tipos de Memória

Cada tipo resolve um problema diferente. O ShopAgent do Dia 2 usa duas delas.

Buffer Memory

RAM DO AGENTE

Janela de contexto atual. Curta duração. O que está na conversa agora.

context.py

```
messages = [
  {"role": "user",
   "content": "Faturamento?"}
]
```

User → AI → User → LIMIT

Sem buffer, o agente esquece a conversa anterior

HOJE

Semantic Memory

HD DO AGENTE

Embeddings em Qdrant. Longa duração. O que o agente "sabe" sobre o domínio.

qdrant_search.py

```
client.search(
  collection="reviews",
  query_vector=embed(question),
  limit=5
)
```

Query → Embed → Qdrant → Top 5

Sem semantic memory, o agente não entende texto

Episodic Memory

DIÁRIO DO AGENTE

Histórico de interações passadas. Aprendizado longitudinal. O que o agente viveu.

history.sql

```
SELECT * FROM conversations
WHERE customer_id = :id
ORDER BY created_at DESC
```

Jan — Fev — Mar — Abr ★

Sem episodic memory, o agente não aprende com o passado

Rafael Rodrigues

SENIOR AI DATA ENGINEER

Brasília, DF • Austin, TX

CONVIDADO ESPECIAL

O dia a dia de um Senior AI Data Engineer

Rafael traz a visão real de quem vive AI Data Engineering no dia a dia — os desafios, as ferramentas, e o que muda na prática quando você adota AI como primitiva.

SKILL SET 2026

FDE + Knowledge Architect

O tradutor de negócio e o arquiteto de conhecimento — juntos, garantem que a IA receba contexto, não texto cru.

ONTOLOGY LAYER

Por Que Ontologia Importa

*Sem ontologia, a IA recebe texto. Com ontologia, a IA recebe **significado** – entidades, relações e vocabulário controlado que transformam busca em compreensão.*

BLOCO 5 · RAG VS LEDGER

As Duas Pernas do Agente

DUAL-STORE STRATEGY

Ledger vs Memory — A Escolha Certa

A pergunta do usuário define o caminho. O agente aprende a rotear.

GUIA DE DECISÃO

Quando Usar Cada Store

*A pergunta do negócio determina a rota. **5 padrões** que cobrem 90% dos casos.*

Faturamento de SP? *Número exato: SUM(total)*	Ledger	`SELECT c.state, SUM(o.total)`
Quem reclama de entrega? *"demorou" = "não chegou" = "15 dias"*	Memory	`qdrant.search("entrega atrasada")`
Ticket médio dos insatisfeitos? *Qdrant acha → Postgres calcula*	Ambos	`qdrant.search() + AVG(total)`
Produtos mais vendidos em março? *Contagem + filtro temporal*	Ledger	`COUNT(*) + WHERE created_at`
Sentimento geral do cliente? *Texto livre, sem campo estruturado*	Memory	`similarity_top_k=10`

“

Um agente inteligente precisa de duas pernas: uma para fatos exatos e outra para significado.

Esta é a arquitetura do ShopAgent. Hoje vocês constroem as duas. Amanhã o agente decide qual usar.

O Fluxo RAG: Como Funciona

Offline (uma vez)

Online (cada query)

O Futuro do RAG — Tipos e Evolução

TIER 3 · DIA 3

Agentic RAG

O agente decide quando recuperar, o que buscar e como combinar RAG + SQL. Totalmente autônomo.

Agent Decides → Qdrant / Postgres → Combine → Respond

↑ ASPIRAÇÃO

TIER 2 · HOJE

Advanced RAG

Pre-retrieval (query expansion) + Post-retrieval (reranking). O que construímos hoje com LlamaIndex.

Query Expand → Embed → Search → Rerank → Respond

↑ EVOLUÇÃO

TIER 1

Naive RAG

Chunk tudo, busca direta, sem refinamento. Ponto de partida — funciona, mas é limitado.

Chunk → Embed → Search → Respond

BLOCO 6 · THE LEDGER

Postgres — A Perna Exata

5 queries canônicas + MCP Supabase conectando Claude ao banco

SCHEMA DO BANCO

Schema do Ledger: 3 Tabelas

Três tabelas normalizadas com UUIDs, foreign keys e check constraints — o mínimo viável para analytics de e-commerce.

VOCABULARIO SQL DO NEGOCIO

5 Queries Canônicas do ShopAgent

01 revenue_by_state
"Faturamento por estado?"

```
SELECT c.state,
       COUNT(*) AS pedidos,
       SUM(o.total) AS faturamento
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
GROUP BY state
ORDER BY faturamento DESC
```

state	pedidos	faturamento
SP	1.230	R$ 127.430
RJ	890	R$ 89.210

02 orders_by_status
"Quantos pedidos por status?"

```
SELECT status,
       COUNT(*) AS total
FROM orders
GROUP BY status
```

status	total
delivered	4.520
shipped	1.230
cancelled	380

03 top_products_by_revenue
"Top produtos por receita?"

```
SELECT p.name,
       SUM(o.total) AS revenue
FROM orders o
JOIN products p
  ON o.product_id = p.product_id
GROUP BY p.name
ORDER BY revenue DESC
LIMIT 10
```

name	revenue
iPhone 15	R$ 42.300
MacBook Air	R$ 38.100

04 payment_distribution
"Distribuição de pagamento?"

```
SELECT payment,
       COUNT(*) AS total,
       ROUND(
         COUNT(*) * 100.0
         / SUM(COUNT(*)) OVER(), 2
       ) AS pct
FROM orders
GROUP BY payment
```

payment	total	pct
pix	3.200	52.1%
credit_card	2.100	34.2%

05 segment_analysis
"Ticket médio por segmento?"

```
SELECT c.segment,
       AVG(o.total) AS avg_ticket,
       COUNT(*) AS pedidos
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
GROUP BY c.segment
```

segment	avg_ticket	pedidos
premium	R$ 487	820
standard	R$ 231	2.940
basic	R$ 98	2.370

*Estas **5 queries são o vocabulário do negócio** em SQL. Toda pergunta de e-commerce cabe numa variação dessas 5. O Claude Code as conhece de cor.*

5 Queries · Business Analytics · SQL Canônico

FLUXO COMPLETO

Ledger em Ação: Pergunta → SQL → Resultado

O caminho determinístico: uma pergunta de negócio vira SQL, executa no Postgres, e retorna um número exato.

BLOCO 7 · THE MEMORY

Qdrant — A Perna Semântica

PIPELINE DE INGESTÃO VETORIAL

LlamaIndex Pipeline: JSONL → Vetores → Qdrant

STAGE 1
Source
gen/data/reviews/reviews.jsonl
203 REVIEWS

→

STAGE 2
Reader
JSONReader(is_jsonl=True)
LLAMAINDEX

→

STAGE 3
Embed
FastEmbed BAAI/bge-base-en-v1.5
768 DIM

→

STAGE 4
Store
QdrantVectorStore(collection='shopagent_reviews')
PERSIST

→

STAGE 5
Index
VectorStoreIndex.from_documents(...)
BUILD

→

STAGE 6
Query
index.as_query_engine(top_k=5)
ONLINE

→ prompts/d2-context/05-llamaindex-pipeline.md

Pergunta Semântica → Contexto → Resposta

A mesma pergunta que antes gerava alucinação — agora retorna dados reais do Qdrant.

Resposta Fundamentada

"Encontrei 23 reviews sobre entrega. Temas: 'demorou 15 dias' (15x), 'ainda não recebi' (5x), 'frete caro' (3x). Sentimento: 78% negativo."

Por que isso importa?

Claude não inventou "demorou 15 dias". Ele leu das reviews reais do Qdrant.

Fundamentação > Alucinação

→ prompts/d2-context/06-rag-query.md

HORA DA DEMONSTRAÇÃO

Demo ao Vivo

ROTEIRO DA DEMO

4 Perguntas, 2 Stores

1 — *"Qual faturamento por estado?"* — Ledger
Pergunta SQL pura — o Ledger responde sozinho

2 — *"Quem reclama de entrega?"* — Memory
Busca semântica — o Memory encontra por significado

3 — *"Ticket médio dos insatisfeitos do Sudeste?"* — Ledger · Memory
*Pergunta híbrida — **as duas pernas juntas***

4 — *"Resumo executivo do ShopAgent"* — Síntese
Claude sintetiza tudo — números + contexto + insight

DEMO HÍBRIDA

As Duas Pernas Juntas

O *Demo* ao Vivo

SETUP EM 2 CONFIGS

```
{ "mcpServers": { "supabase": { "type": "http", "url": "http://localhost:54321/mcp" },
"qdrant": { "command": "npx", "args": ["-y", "@qdrant/mcp-server"] } } }
```

*Claude Code agora tem acesso **às duas pernas** via MCP oficial. Nenhuma linha de integração customizada.*

NA PRÁTICA:

*Um arquivo JSON configura tudo. O MCP é o **protocolo universal** de conexão entre LLM e ferramentas.*

DEMO FLOW

1. *"Qual faturamento por estado?"* — Ledger responde
2. *"Quem reclama de entrega?"* — Memory responde
3. *"Ticket médio dos reclamantes?"* — Hybrid
4. *"Resumo executivo do ShopAgent"* — Claude sintetiza

NA PRÁTICA:

*Cada prompt escala em complexidade. O aluno começa com **uma perna** e termina com as **duas juntas**.*

*O **Claude Code com MCP** é o agente do Dia 2. Ainda é humano quem faz as perguntas — amanhã o agente vai **fazer as perguntas sozinho**.*

MCP Oficial | 2 Stores | Live Demo

BLOCO 8 · ENCERRAMENTO

O Que Vocês Construíram Hoje

“Sua IA agora pesquisa nos seus dados antes de responder. Nunca mais vai inventar.”

*Vocês saem desta noite com **o Ledger indexado**, **o Memory pronto**, e os dois **conectados via MCP**. Amanhã, o agente vai fazer as perguntas sozinho — e decidir qual perna usar.*

- ✓ Prompt Eng → Context Eng entendido
- ✓ LlamaIndex + Qdrant indexando reviews reais
- ✓ MCP Supabase + MCP Qdrant conectados
- ✓ Claude Code pesquisando nos dois stores